

FloatChat Database Architecture Guide

The Question: Merge Profile Data or Keep Separate?

Problem: You have 15 large profile CSV files from different ARGO floats. Should you merge them into one giant table or keep them separate for optimal chatbot performance?

Answer: NO, don't merge them into one giant table! Here's the right architecture:

Smart Architecture for Chatbot Queries

Option 1: Keep Separate Profile Tables (RECOMMENDED)

```
sql

-- One table per float
profiles_1234567.. -- float 1234567 data
profiles_2345678.. -- float 2345678 data
profiles_3456789.. -- float 3456789 data
...

-- Master index table for chatbot
CREATE TABLE profile_index (
  ... float_id VARCHAR(20),
  ... table_name VARCHAR(50),
  ... record_count INTEGER,
  ... date_range_start TIMESTAMP,
  ... date_range_end TIMESTAMP,
  ... lat_min REAL, lat_max REAL,
  ... lon_min REAL, lon_max REAL,
  ... parameters TEXT[] -- ['temperature', 'salinity', 'pressure']
);
```

Option 2: Smart Partitioning (ADVANCED)

```
sql
```

```
-- Single table but partitioned by float_id
CREATE TABLE profiles (
  profile_id SERIAL,
  float_id VARCHAR(20),
  profile_date TIMESTAMP,
  latitude REAL,
  longitude REAL,
  -- other columns
) PARTITION BY HASH (float_id);

-- Creates 15 physical partitions automatically
```

How Chatbot Will Work

User Query Example: "Show me temperature profiles for float 1234567 in March 2023"

Chatbot Logic:

- 1. **Parse query** → Extract: float_id=1234567, parameter=temperature, time=March 2023
- 2. **Check index table** → Find which table/partition has this data
- 3. **Query specific table** → `SELECT * FROM profiles_1234567 WHERE...`
- 4. **Return focused results** → Much faster than scanning millions of rows

Why This Architecture Works Better

Approach	Query Speed	Storage	Chatbot Complexity
One Giant Table	❌ Slow	❌ Bloated	❌ Complex indexes
Separate Tables	✅ Fast	✅ Organized	✅ Simple routing
Partitioned Table	✅ Fast	✅ Efficient	✅ Transparent

Recommended Implementation

python

```

# Chatbot backend will do this:
def process_query(user_query):
    # 1. Parse: "temperature data for float 1234567"
    parsed = parse_natural_language(user_query)

    # 2. Route to correct table
    if parsed.float_id:
        table_name = f"profiles_{parsed.float_id}"
    else:
        # Query multiple tables based on location/time
        table_name = get_relevant_tables(parsed.location, parsed.time)

    # 3. Execute targeted query
    sql = build_sql_query(parsed, table_name)
    results = execute_query(sql)

    return results

```

✓ Final Recommendation

Go with separate tables approach:

1. Keep your 15 profile CSV files as 15 separate tables
2. Create a lightweight index/metadata table
3. Chatbot queries the right table(s) based on user input

Benefits:

- ⚡ **Lightning fast queries** - No need to scan millions of rows
- 💡 **Easy for LLM to understand** table routing logic
- 🛠️ **Simple to maintain and extend** - Add new floats easily
- 📊 **Each float's data stays organized** - Natural separation

Implementation Steps:

1. Create 15 separate profile tables: `profiles_[float_id]`
2. Create master index table with metadata summaries
3. Build chatbot query router that selects appropriate table(s)
4. LLM generates targeted SQL for specific tables

Database Schema Example

```
sql

-- Example for float 1234567
CREATE TABLE profiles_1234567 (
  profile_id SERIAL PRIMARY KEY,
  cycle_number INTEGER,
  profile_date TIMESTAMP,
  latitude REAL,
  longitude REAL,
  pressure REAL,
  temperature REAL,
  salinity REAL,
  depth REAL,
  temp_qc INTEGER,
  sal_qc INTEGER,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Master index for chatbot routing
CREATE TABLE profile_index (
  float_id VARCHAR(20) PRIMARY KEY,
  table_name VARCHAR(50),
  record_count INTEGER,
  date_range_start TIMESTAMP,
  date_range_end TIMESTAMP,
  lat_min REAL, lat_max REAL,
  lon_min REAL, lon_max REAL,
  parameters TEXT[],
  last_updated TIMESTAMP DEFAULT NOW()
);
```

This architecture will make your FloatChat chatbot much more responsive and easier to maintain!

Next Steps:

1. Set up the 15 separate profile tables
2. Populate the profile_index table
3. Build the chatbot query routing logic

4. Test with sample natural language queries